Hierarchical Device–Edge–Cloud Task Offloading: An Experimental Study of DQN, Double DQN, Feedback, and Cloud Scheduling

Tasnim Mizaoui, Farah Cherif, and Doua Bousnina
INSAT, Tunis, Tunisia

***Abstract*—This paper presents an experimental framework for hierarchical device–edge–cloud task offloading. It integrates centralized DQN, hierarchical DQN, and hierarchical Double DQN controllers with causal adjacent-tier feedback in a common SimPy backend, where communication bytes, delay, and information age are modeled explicitly. The matched campaign replays 579 chronologically held-out tasks derived from Alibaba Cluster Trace 2017 after independent-task filtering; 2,891 accepted tasks are split into 1,734 training, 578 validation, and 579 test tasks and mapped to 24 devices, 3 edges, and one shared cloud worker. In the reference-topology comparison, centralized DQN reaches the highest mean deadline success (77.41%), Greedy/EDF the lowest mean P95 latency (12.95 s), and Greedy/FIFO and Greedy/EDF the lowest mean device energy (0.211 J/task). Hierarchical Double DQN improves P95 latency and device energy relative to hierarchical DQN, with a small decrease in success. Separate feedback experiments show roughly 70% less feedback traffic with event-triggered updates at a latency cost, while the cloud ablation favors fixed SPT. The framework makes these centralized–hierarchical–heuristic trade-offs measurable and provides a reproducible basis for future sequential multi-agent studies.**

***Index Terms*—mobile edge computing, task offloading, hierarchical reinforcement learning, discrete-event simulation, Double-DQN, reproducibility**

I. Introduction

Multi-access edge computing (MEC) places compute close to data producers while retaining access to larger cloud or HPC resources [1]. A task must therefore be mapped to a device CPU, an associated edge CPU, or a shared cloud queue. The choice couples execution time, queueing, wireless and backhaul transmission, device energy, and a relative deadline. A centralized controller can observe more state, but it also requires broader information sharing and can become a coordination bottleneck.

We study these choices under common simulator conditions. Each tier makes only the decision exposed by its local contract: the device chooses local execution or its assigned edge, the edge chooses edge execution or cloud forwarding, and the cloud selects a queue discipline. A common SimPy backend executes the centralized and hierarchical policies with shared task, timing, energy, and metric definitions, making cross-policy differences directly inspectable.

We ask how hierarchical policies compare with centralized and heuristic alternatives, how much control traffic event-triggered feedback saves, and whether a learned cloud selector improves on fixed scheduling rules. To answer these questions, we replay 579 held-out Alibaba-derived tasks using 24 devices, three edges and one shared worker. We use paired replays to isolate the cloud scheduler and measure feedback traffic separately from application data. Section II reviews related work, Section III defines the model, and Sections IV and V present the evaluation and conclusions.

II. Related Work

ETSI defines MEC as a distributed service environment close to the access network [1]. Offloading literature consequently studies coupled placement, queueing, wireless transmission, energy, and deadline decisions; the communication–energy trade-off is surveyed in [2]. This paper narrows the decision boundary deliberately: device, edge, and cloud controllers receive different local observations and expose small discrete action spaces rather than solving one global optimization problem.

Simulation frameworks make different abstraction choices. EdgeCloudSim provides configurable edge-network and compute models [3]; iFogSim models resource management in IoT/fog workflows [4]; and PureEdgeSim targets cloud, edge, and mist scenarios with broad extensibility [5]. The present implementation uses a compact SimPy model so that policy injection and the three decision boundaries remain explicit. This choice supports paired policy replays in the same event model. The specialized frameworks offer richer mobility, application and network models for studies where these factors are central.

Deep Q-learning established value-based control from replayed transitions [6], while Double DQN separates action selection and target evaluation to reduce overestimation [7]. Hierarchical and multi-agent MEC studies generally decompose placement or resource allocation across levels; federated reinforcement learning is a related future direction because it can distribute training without centralizing raw local experience. Hierarchical deep RL for joint caching and offloading similarly separates decisions across levels [8]; federated deep RL aggregates policies learned from distributed environments [9]; and centralized-training/decentralized-execution methods expose the information asymmetry that

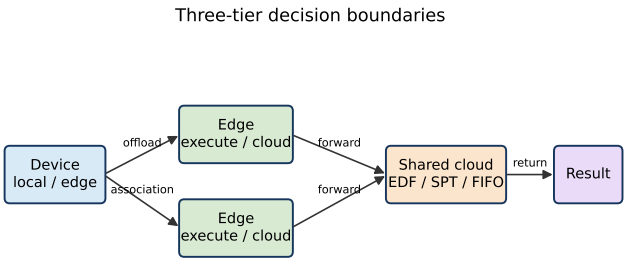


Fig. 1. Implemented device–edge–cloud decision boundaries.

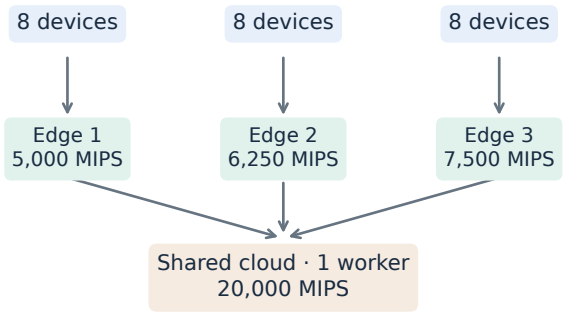


Fig. 2. Reference RL topology: 24 devices, three associated edges, and one shared cloud worker.

motivates local policy boundaries [10]. These approaches often assume sequential environments, richer multi-agent interaction, or explicit model aggregation. Our study compares value-based controllers under matched workloads and separates feedback traffic and cloud scheduling through targeted ablations.

Alibaba Cluster Trace 2017 supplies production datacenter arrival and compute records [11]. It is not a mobile-MEC measurement. The adapter derives computational work from completed instances, while payloads, deadlines, device assignment, battery, and wireless links are declared simulation assumptions.

III. Architecture and Proposed Approach

A. Three-tier architecture

The system has device, edge, and cloud decision levels. A device executes locally or sends a task to its fixed associated edge. The edge executes the task or forwards it to a shared cloud scheduler. The cloud chooses a queue discipline when a worker becomes available. The small simulator demonstration uses three devices and two edges; the RL campaign instead uses 24 devices, three edges, and one shared cloud. Device-to-edge associations are explicit, not learned as arbitrary routing choices. The campaign association is round-robin across the three edges, as shown in Fig. 2. The decision boundaries are shown in Fig. 1.

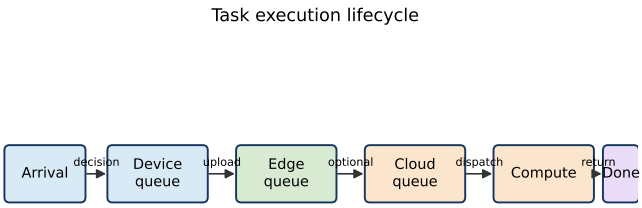


Fig. 3. Task lifecycle through queues, links, execution, and result return.

B. Task lifecycle and resource model

The lifecycle is arrival, device decision, optional device–edge upload, edge decision, optional edge–cloud upload, cloud dispatch, execution, and result return. Queues contribute waiting time, and every modeled link contributes serialization, propagation, and resource queue delay. Fig. 3 illustrates these paths.

Each task has arrival time, decimal-MB input and result sizes, required MI, device identifier, relative deadline, and priority. For demand w MI and capacity c MIPS,

$$t_{\text{cpu}} = w/c. \tag{1}$$

For payload s decimal MB, bandwidth B Mbps, propagation p, and queue wait q,

$$t_{\text{link}} = 8s/B + p + q. \tag{2}$$

Latency is completion minus arrival, and a task meets its deadline when $t_{\text{completion}} \leq t_{\text{arrival}} + D$. Devices and edges use capacity-one SimPy resources. The current cloud has one worker and chooses EDF, shortest processing time (SPT), or FIFO. Device energy includes CPU energy per MI and transmit/receive power times radio duration; idle, reception-infrastructure, retransmission, and cloud energy are not modeled.

C. Observations and actions

The device observation contains normalized work, input size, deadline slack, battery fraction, device capacity, device–edge link state, and an optional delayed edge-congestion signal. Its actions are local execution and edge offloading. The edge observes task demand and deadline, edge queue/utilization, edge capacity, edge–cloud link state, and an optional delayed cloud signal. Its actions are edge execution and cloud forwarding. The cloud observes queue length, deadline summaries, estimated service time, oldest wait, the count of near-deadline jobs and workload imbalance. Device, edge and cloud use 11, 10 and 11 features. Work is log-scaled to compress its heavy tail. The cloud chooses EDF (earliest absolute deadline), SPT (shortest estimated service), or FIFO (first arrival to the cloud queue). A centralized comparator uses a 16-feature global observation and selects device, edge, or cloud, with

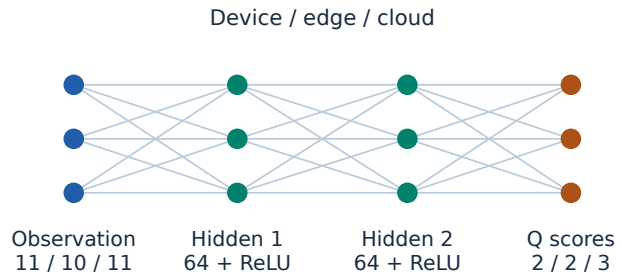


Fig. 4. DQN architecture shared by design across the three tiers. Each controller has its own learned weights and input/output sizes.

fixed EDF cloud dispatch. Lower-tier hierarchical agents do not receive raw global state.

D. Learning and reward

Q-learning estimates how much reward an agent can expect from an action and the decisions that follow. For a policy π, the action value is

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^{k} r_{t+k} \mid s_t = s,\ a_t = a\right]. \tag{3}$$

Here s is the observation used for control, a is the action, and r_t is the reward assigned to the current decision. A tabular Q-learning update adjusts the value of the observed pair toward a target that combines the immediate reward and the best estimated next action [12].

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{b} Q(s',b) - Q(s,a)\right]. \tag{4}$$

The learning rate α controls the correction. At episode termination the future term is zero.

Our observations include continuous quantities such as remaining deadline, battery and queue wait. A table would require discretizing their combinations. Deep Q-learning approximates $Q(s,a)$ with a neural network. A Deep Q-Network (DQN) returns one score per discrete action in a single pass. Fig. 4 shows the architecture used at each tier. The controller explores during training and chooses the highest-scoring action during evaluation.

The DQN and Double DQN agents use two hidden layers of 64 units, replay capacity 8,000, batch size 32, learning rate 3×10^{-4}, target updates every 500 optimization steps, and Smooth L1 loss. Double DQN selects the next action with the online network and evaluates it with the target network. DQN takes the target-network maximum. Splitting selection and evaluation addresses the upward bias that can arise when choosing the largest of several noisy estimates [7].

For non-cloud decisions, the shared task reward is

$$R = -\Bigg(w_l \min\left(1, \frac{\log(1+L/D)}{\log(10001)}\right) + w_e \min\{3, E/E_0\} + w_d \mathbb{1}[\text{miss}] + w_c \min\{3, F/F_0\}\Bigg), \tag{5}$$

with $(w_l, w_e, w_d, w_c) = (0.35, 0.25, 0.35, 0.05)$. Here L, D, E, and F denote end-to-end latency in seconds, deadline in seconds, device energy in joules, and feedback bytes. The implementation uses $E_0 = 0.3$ J and $F_0 = 64$ bytes from the default `RewardWeights` configuration; these are normalization references, not measured campaign means. The miss indicator is one when a deadline-feasible task misses its deadline. These weights are design choices that trade off latency, device energy, feasible deadline misses and feedback cost in a common scalar. The logarithm compresses large latency ratios. In $\log(10001)$, the number 10,001 is $1 + 10{,}000$. At $L/D = 10{,}000$, the normalized latency penalty reaches one, and clipping caps it there. For example, with $L = 2$ s, $D = 1$ s, $E = 0.15$ J, $F = 32$ B and a nominally feasible deadline, the four weighted penalties are about 0.042, 0.125, 0.350 and 0.025. The reward is their negative sum, approximately -0.542. The cloud uses a queue-waiting-cost reward for contested dispatches, so its objective is not identical to the shared task reward.

The task reward is computed after completion and assigned to the decisions on its route. The evaluated protocol links consecutive observations in each controller's decision log, with $\gamma = 0.9$ and a terminal transition at the end of the log. Consecutive decisions can concern different tasks. For reward r and next observation s', DQN uses $y = r + \gamma \max_a Q_{\text{target}}(s', a)$. Double DQN selects $a^* = \arg\max_a Q_{\text{online}}(s', a)$ and uses $y = r + \gamma Q_{\text{target}}(s', a^*)$. Both use $y = r$ at the final transition.

The cloud reward is $-\min\{1, \log(1 + n\hat{t})/\log(10001)\}$, where n is the queue size at dispatch and $\hat{t}$ is the estimated service duration in seconds. This proxy for queue occupancy during service favors shorter jobs at a given queue size. Singleton queues are omitted from cloud replay because the three rules select the same job.

E. Feedback

Feedback is adjacent-tier and causal. A snapshot sampled at time t is available only after modeled propagation and serialization, at a later decision. Conditions are none, periodic, frequent, and event-triggered. The event trigger sends an initial snapshot, sends when the signal change reaches a threshold, and refreshes after five observations. Feedback bytes, modeled delay, and information age are recorded. At a decision, information age is the elapsed time since the consumed snapshot was sampled [13]. The modeled control path operates independently of data-plane queues. Fig. 5 shows this control path.

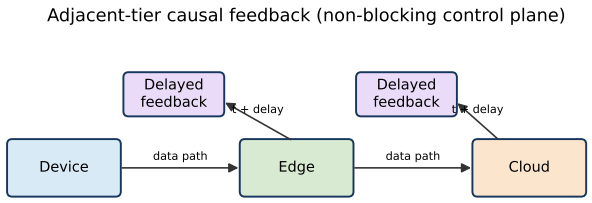


Fig. 5. Causal adjacent-tier feedback with a non-blocking control plane.

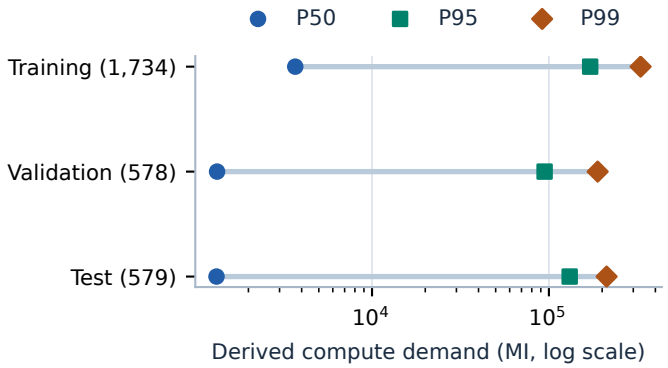


Fig. 6. Compute demand after trace adaptation. Each row reports the median, P95 and P99 for one chronological split.

IV. Implementation and Performance Evaluation

A. Software and reproducibility

The implementation targets Python 3.11 and uses SimPy for event scheduling, PyTorch for learning, and NumPy, pandas, Matplotlib and PyYAML for workload preparation and analysis. For the six-policy comparison, we matched task identifiers and arrival timestamps, verified the trace and topology fingerprints, and checked that training, validation and evaluation noise seeds were disjoint. The resulting dataset contains 52,110 task outcomes and 90 replay summaries. A 20-task pilot with each algorithm verified that all policies ran on the same reference configuration.

B. Dataset and workload construction

Alibaba Cluster Trace 2017 records a 24-hour period from part of a production cluster that runs batch jobs and online services [11]. We use the batch task and instance tables. The task table supplies creation time and the job and task identifiers. The instance table supplies start and finish timestamps and average CPU use in cores. We join the two tables by job and task identifiers.

We retain completed tasks belonging to single-task jobs with at least one valid completed instance. For task i, the work in million instructions (MI) is derived as

$$w_i = \frac{c_{\mathrm{ref}}}{|\mathcal{I}_i|} \sum_{j \in \mathcal{I}_i} (t_j^{\mathrm{end}} - t_j^{\mathrm{start}}) u_j, \tag{6}$$

where $\mathcal{I}_i$ contains valid completed instances, u_j is average CPU use and $c_{\mathrm{ref}} = 1000$ MIPS per core is a modeling constant. The average represents a typical instance. Summing across parallel instances would map their combined work onto one sequential service. The filter retains 2,891 of 80,553 task records. In the separate instance table, 147,849 records are flagged for malformed fields or invalid completed-instance measurements.

The accepted tasks are sorted by arrival and split chronologically into 1,734 training, 578 validation and 579 test tasks. Fig. 6 shows the compute-demand quantiles by split.

The accepted test tasks span 9,753 trace seconds. Their median work is 1,320 MI and their P95 work is 131,057 MI. The six largest tasks account for 14.5% of the test workload. A P95-sized task takes about 6.55 s of nominal service even at 20,000 MIPS. A single worker can therefore remain occupied while several smaller jobs arrive.

The evaluation uses the best-tier-calibrated deadline rule with a one-second floor and validation-calibrated arrival compression 0.12793575549650835. Payload sizes, device assignment, device battery and radio state, wireless bandwidth and propagation, edge/cloud capacities, and relative deadlines are simulator assumptions. Input size is assigned reproducibly per task using a lognormal distribution with median 1 MB, log-scale standard deviation 1 and a cap of 50 MB. The result returned to the device is 0.1 MB. Tasks are assigned round-robin to 24 devices with fixed edge associations. The test median interarrival time changes from 10 s in the trace to 1.28 s after compression. The dataset therefore supplies workload timing and compute demand, while the simulator defines the mobile network and deadline regime.

C. Matched evaluation protocol

The primary comparison uses 579 tasks on the reference topology: 24 devices, three edges, and one cloud worker. Random, Greedy/FIFO, and Greedy/EDF are fixed baselines evaluated with noise seeds 10001–10005. Centralized DQN uses the five standard-DQN checkpoints from training seeds 42–46, observes global state, and uses fixed EDF cloud dispatch. Hierarchical DQN and hierarchical Double DQN use the ten frequent-feedback bundles at the same five training seeds; their cloud rules are learned. For learned policies, five noise runs are averaged within each training seed, then the mean and sample standard deviation are computed across five training-seed means. Baseline deviations are across the five noise seeds. P95 is computed per replay before aggregation, so these two SDs describe different variation sources.

Table I lists the main settings. Checkpoint selection maximizes validation deadline success and uses lower P95 to break ties. Training stops early after eight validation checks without improvement for both learners. For the test,

TABLE I
Reference simulation and training settings.

Parameter	Setting
Compute capacity	Device 800/1,000/1,200 MIPS, edge 5,000/6,250/7,500 MIPS, cloud 20,000 MIPS
Device–edge link	100 Mb/s, 10 ms propagation
Edge–cloud link	1,000 Mb/s, 40 ms propagation
Timing noise	Bandwidth and service perturbations of up to 10%
Relative deadline	$\max\{1, 0.9w/20000 + 0.5\}$ s, with w in MI
Training	Up to 1,000 episodes of 100 tasks, nine training topologies
Exploration	Linear from 0.90 to 0.05 over the first 75% of the budget
Validation	578 tasks, five noise seeds, every 25 episodes

TABLE II
Matched reference-topology comparison. Means ± SD; P95 is per replay before aggregation.

Policy	Deadline success (%)	P95 latency (s)	Device energy (J/task)
Random	49.57 ± 0.95	137.96 ± 16.46	10.480 ± 0.720
Greedy/FIFO	75.79 ± 0.78	13.62 ± 0.49	0.211 ± 0.001
Greedy/EDF	75.89 ± 0.86	12.95 ± 0.43	0.211 ± 0.001
Centralized DQN	77.41 ± 1.37	15.71 ± 4.44	0.242 ± 0.067
Hierarchical DQN	74.63 ± 0.79	16.91 ± 3.32	0.260 ± 0.104
Hierarchical Double DQN	74.33 ± 1.66	15.08 ± 2.85	0.244 ± 0.065

the original deadline formula stays fixed when changing topology.

D. Matched six-policy comparison

Table II is the primary result. Device energy is reported in joules per task. Reported means and standard deviations summarize the observed replays.

Figure 7 visualizes the same documented summary. Centralized DQN has the highest mean success, while Greedy/EDF has the lowest mean P95. Greedy/FIFO and Greedy/EDF share the lowest mean energy. Both hierarchical learners improve on Random across all three headline metrics, while the reference comparison exposes different strengths for centralized control, heuristic scheduling, and hierarchy. Double DQN lowers mean P95 and energy relative to hierarchical DQN, with a small success decrease. These are descriptive comparisons, not claims of statistical significance.

Fig. 8 shows why the P95 matters. Hierarchical DQN has a mean latency of 3.04 s and a mean per-replay P95 of 16.91 s. The high tail includes queuing, execution and transfers. It indicates a risk for latency-sensitive IoT applications under this compute-heavy workload and modeled contention.

a) Additional centralized control: The primary table uses one centralized reference. As an additional control, we evaluated the five centralized Double DQN checkpoints under the same trace, reference topology, and five noise seeds. This variant uses fixed EDF in the cloud and obtains $77.40 \pm 1.24\%$ deadline success, 13.74 ± 2.08 s P95, and 0.224 ± 0.026 J/task. The two centralized variants have nearly identical mean success. Centralized Double DQN has a lower mean P95, while Greedy/EDF retains the lowest mean P95 of the evaluated policies. The centralized control separates the choice of learning target from the choice of information and control architecture, although centralized and hierarchical cloud dispatch differ.

TABLE III
Mean latency and communication. Means ± SD. Data and feedback volumes are per 579-task replay.

Policy	Mean L (s)	Data (10^8 B)	Feedback (10^3 B)
Random	22.002 ± 2.681	8.07 ± 0.57	0
G/FIFO	2.619 ± 0.046	12.86 ± 0.12	0
G/EDF	2.472 ± 0.088	12.86 ± 0.12	0
C-DQN	2.794 ± 0.447	12.54 ± 0.68	0
H-DQN	3.045 ± 0.292	12.72 ± 1.08	35.4 ± 0.7
H-DDQN	3.015 ± 0.470	12.04 ± 0.42	35.1 ± 0.6

TABLE IV
Coordination trade-off on the reference topology.

Learner	Mode	Success (%)	P95 (s)	Feedback (kB)
DQN	Frequent	74.63	16.91	35.39
DQN	Event 0.10	75.74	18.84	11.86
DQN	Event 0.25	73.07	16.61	11.99
DQN	Event 0.50	74.75	16.74	10.69
Double DQN	Frequent	74.33	15.08	35.13
Double DQN	Event 0.10	74.29	17.00	11.53
Double DQN	Event 0.25	74.95	17.46	11.09
Double DQN	Event 0.50	74.98	17.80	10.43

E. Feedback and topology sensitivity

To isolate feedback settings, each algorithm was evaluated with five training seeds under each condition. Table IV compares frequent updates with event-triggered updates at the checkpoint's trained threshold. Traffic is measured per 579-task replay.

At threshold 0.50, Double DQN reduces feedback traffic by 70.3% relative to frequent feedback, with a 0.65 percentage point increase in observed success and a 2.72 s increase in P95. For DQN, the same comparison reduces feedback bytes by 69.8%, with a 0.12 point increase in observed success and a 0.17 s decrease in P95. These comparisons use separately trained policies and the cost includes feedback bytes. They describe the combined effect of the feedback mechanism and learning under that mechanism.

Four variants probe transfer to resource settings held out during training. They increase cloud CPU, improve the device–edge link, improve the edge–cloud link, or combine reduced CPU and link capacity with longer propagation. Under combined stress and frequent feedback, DQN reaches 61.73% success and 29.42 s P95, compared with 60.17% and 40.41 s for Double DQN. The reference latency ranking reverses in this condition. The matched evaluation therefore supports conclusions tied to a workload and topology.

F. Cloud scheduling with frozen upstream policies

The cloud ablation replays each frequent-feedback checkpoint with the learned rule selector and each fixed rule,

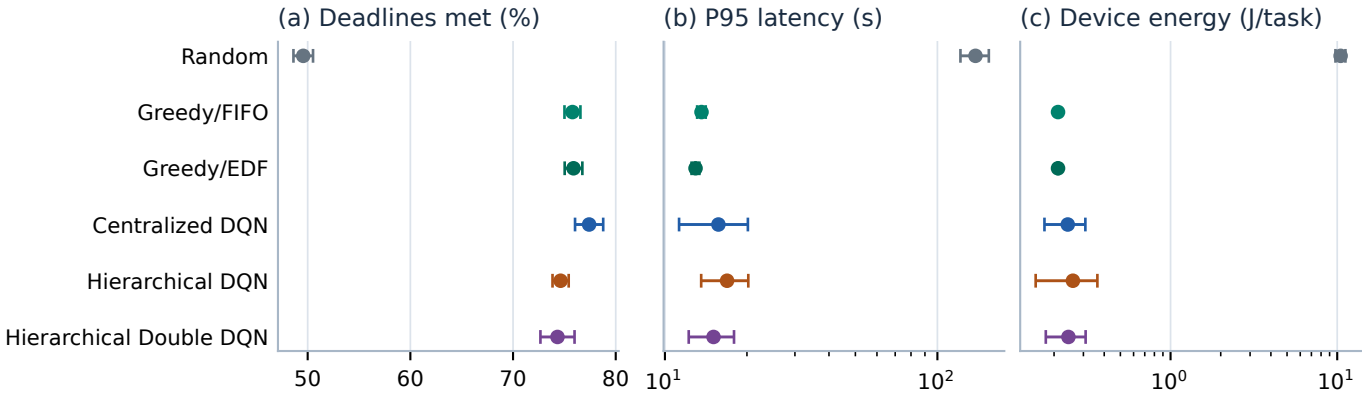


Fig. 7. Matched-policy comparison. Points are means and whiskers are one sample standard deviation across training-seed means for learned policies or across noise seeds for fixed policies. Latency and energy use logarithmic axes.

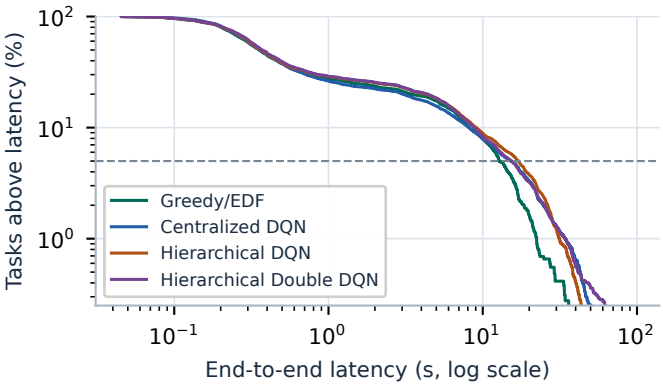


Fig. 8. Empirical latency exceedance for four policies. The dashed line marks 5%. The curves pool the same held-out tasks over the available replays per policy. Table II averages per-replay P95 values.

TABLE V
CLOUD ABLATION ON THE REFERENCE TOPOLOGY.

	DQN upstream		Double DQN upstream	
Rule	Success (%)	P95 (s)	Success (%)	P95 (s)
Learned	74.63	16.91	74.33	15.08
EDF	74.65	16.37	74.24	15.29
SPT	75.23	12.26	74.82	12.36
FIFO	74.25	18.25	73.90	17.82

keeping device and edge weights, tasks and noise seeds paired. The results in Table V show the effect of switching the cloud scheduler under the same upstream policies.

SPT gives the highest mean success and lowest mean P95 for both upstream architectures across the five tested topologies. With the Double DQN upstream policy on the reference, SPT reduces P95 by 2.72 s relative to its learned scheduler. All ten checkpoints consistently select one rule whenever multiple jobs are queued, and the corresponding fixed-rule replay reproduces their per-task outcomes. This explains why evaluating against all three fixed rules matters. An improvement over EDF alone could reflect the choice of a different fixed rule. The long tail of task work and the short-service preference in the cloud reward help explain the competitive SPT result.

G. Reporting limitations

The evaluation reports mean latency, P95 latency, deadline success, device energy, task communication bytes, and feedback bytes; Table III adds the metrics not included in Table II. Device energy is total modeled device CPU and radio energy divided by the tasks in a replay, then aggregated according to the seed aggregation described above. It excludes idle, reception-infrastructure, retransmission, and cloud energy. Task communication bytes are data-plane payload bytes; feedback bytes are control-plane bytes. Feedback currently does not contend with data-plane links, so its communication cost should not be interpreted as queueing interference.

Centralized DQN observes global state and therefore has an information advantage over the hierarchical policies; centralized state-collection traffic is not included in the communication accounting. The five evaluation seeds vary simulated bandwidth and service-time noise on one chronological test window, not five independent trace periods, and the five training seeds provide limited evidence. The reported standard deviations are descriptive and reflect different aggregation sources for fixed and learned policies.

V. Conclusion and Perspectives

The three-tier framework makes placement, cloud scheduling and coordination measurable under paired task replays. On the reference topology, centralized DQN reaches 77.41% deadline success, and the additional centralized Double DQN control reaches 77.40%. Greedy/EDF achieves the lowest mean P95, at 12.95 s. Among hierarchical controllers, Double DQN reduces the reference P95 from 16.91 to 15.08 s relative to DQN, with a 0.30 percentage point decrease in deadline success. The latency ranking reverses under combined stress.

The frozen-policy ablation identifies SPT as the strongest cloud rule on the tested workloads. On the reference, replacing the learned Double DQN cloud scheduler by SPT reduces mean P95 from 15.08 to 12.36 s with the same upstream weights. The event-triggered feedback experiment also identifies a practical control mechanism. At threshold 0.50, Double DQN transmits about 70% fewer feedback bytes than with frequent updates, with a 2.72 s increase in mean P95.

These results characterize one chronological test window and a single-worker cloud. An independent trace period, multi-worker execution and a shared data/control network would extend the evaluation. Federated training is a further direction. Devices could aggregate local updates at each edge before the edges synchronize at the cloud, with synchronization traffic measured separately from inference feedback.

References

[1] ETSI, "Multi-access edge computing (mec); framework and reference architecture," ETSI GS MEC 003 V2.2.1, 2020. [Online]. Available: https://www.etsi.org/deliver/etsi_gs/MEC/001_099/003/02.02.01_60/gs_MEC003v020201p.pdf

[2] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Communications Surveys and Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017.

[3] C. Sonmez, A. Ozgovde, and S. Ergen, "EdgeCloudSim: An environment for performance evaluation of edge computing systems," in *2017 Second International Conference on Fog and Mobile Edge Computing*, 2017, pp. 39–44.

[4] H. Gupta, A. Vahid Dastjerdi, S. K. Ghosh, and R. Buyya, "iFogSim: A toolkit for modeling and simulation of resource management techniques in the internet of things, edge and fog computing environments," *Software: Practice and Experience*, vol. 47, no. 9, pp. 1275–1296, 2017.

[5] C. Mechalikh, H. Taktak, and F. Moussa, "PureEdgeSim: A simulation framework for performance evaluation of cloud, edge and mist computing environments," *Computer Science and Information Systems*, vol. 18, no. 1, pp. 43–66, 2021.

[6] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[7] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, 2016.

[8] C. Sun, X. Li, C. Wang, Q. He, X. Wang, and V. C. M. Leung, "Hierarchical deep reinforcement learning for joint service caching and computation offloading in mobile edge-cloud computing," *IEEE Transactions on Services Computing*, vol. 17, no. 4, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10409617

[9] H. H. Zhuo, W. Feng, Y. Lin, Q. Xu, and Q. Yang, "Federated deep reinforcement learning," arXiv:1901.08277, 2019. [Online]. Available: https://arxiv.org/abs/1901.08277

[10] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," arXiv:1706.02275, 2017. [Online]. Available: https://arxiv.org/abs/1706.02275

[11] Alibaba Group, "Alibaba Cluster Trace 2017," Alibaba Cluster Trace Program, 2018. [Online]. Available: https://github.com/alibaba/clusterdata/blob/master/cluster-trace-v2017/trace_201708.md

[12] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 1992.

[13] R. D. Yates and S. K. Kaul, "The age of information: Real-time status updating by multiple sources," *IEEE Transactions on Information Theory*, vol. 65, no. 3, pp. 1807–1827, 2019. [Online]. Available: https://arxiv.org/abs/1608.08622